

AgentDescent: Gradient Descent Where the Parameters Are Agents

A Parallel, Asynchronous, Merge-Based Framework for Self-Evolving Agent Artifacts

Birfy Chen

birfychen@gmail.com

<https://github.com/Birfy/agentdescent>

August 2026

Abstract

Self-improving agent systems — agents that evolve their own skills, prompts, and harnesses — are today overwhelmingly *serial*: one candidate is proposed, evaluated, and accepted or rejected per iteration, bounding improvement throughput at $O(1/T_{\text{iter}})$. We present **AgentDescent**, a framework that transplants the deep-learning *training stack* onto agent self-evolution. The “parameters” are a library of evolvable artifacts (skills, prompts, harness modules, verifiers); the “gradients” are *diffs carrying evidence cards*; the “optimizer step” is a *merge decision* over a git-backed, version-vectored ledger. N workers propose diffs in parallel and a barrier-free asynchronous aggregator merges them, targeting $O(N/T_{\text{iter}})$ improvement throughput. The one place the analogy must break defines the system: *gradients add, diffs do not* — so aggregation is not averaging but conflict resolution, statistical acceptance under a per-artifact Beta posterior, and transactional commit. Speed comes from three levers: *overlapping I/O-bound rollouts*; *removing the round barrier* (staleness becomes a first-class per-diff lag η , governed by a version-lag budget and Full / Guarded / Reflective policies); and — the lever specific to merge-based evolution — *validating less*: held-out evaluation is the critical path, and merging fuses N concurrent proposals into *one* gate sweep, with *reflective merge* (a model writes the union of contradicting proposals; the gate, not the model, decides) extending the saving to single-key artifacts. Published algorithms adapt by writing plug-ins, not forks: a *strategy* that encodes their artifact as keyed diffs and, where needed, their own optimizer behind *aggregator_factory* — eighteen ports (seven benchmark-faithful: ACE, GEPA, EvoSkill, SkillOpt, ADAS, DGM, OpenEvolve; eleven declared microports and analogues) run unchanged under serial, synchronous, and barrier-free scheduling. On a live model (GEPA/HotpotQA), the same four workers reach $1.85\times$ effective concurrency with the barrier and $3.22\times$ without it, against a true serial control at a pinned budget; merging cuts model calls 28% at width 8. An equal-budget merge-vs-fork comparison is reported honestly as not yet separated. Code and every number’s reproduction command are open source.

1 Introduction

A rapidly growing literature shows that large-language-model agents can improve their own artifacts: their prompts [1, 3], their in-context playbooks [28], their skill libraries [20, 30], their workflows [5, 27], and even their own code [13, 14, 24, 26]. Nearly all of these systems share an execution shape inherited from their conceptual ancestor, the Gödel machine [16]: a *serial* improvement loop. One candidate modification is proposed, evaluated against a benchmark, and accepted or rejected; then the next. If one iteration takes T_{iter} wall-clock seconds, improvement throughput is bounded at one accepted change per T_{iter} — regardless of how much compute is available.

Deep learning faced the same bottleneck a decade ago and solved it with a now-standard stack: data-parallel workers computing gradients concurrently, parameter servers aggregating them [9], lock-free and

staleness-tolerant asynchronous updates [12], adaptive per-parameter step sizes [8], and gradient surgery when updates conflict [25]. The premise of this paper is that the same stack, suitably reinterpreted, applies to agent self-evolution.

AgentDescent realizes this premise (Fig. 1). The parameter tensor becomes a *library of evolvable artifacts*; the gradient becomes a *diff* annotated with an *evidence card* recording the rollout that motivated it; the parameter server becomes a git-backed, version-vectored *ledger*; and the optimizer step becomes an *aggregator* that merges concurrent diffs. N workers roll out tasks against a snapshot of the library and propose diffs in parallel; a barrier-free asynchronous merger aggregates them into a shared, version-controlled artifact library, targeting $O(N/T_{\text{iter}})$ improvement throughput where the serial loop is bounded at $O(1/T_{\text{iter}})$.

The analogy is productive precisely because of where it breaks. *Gradients add; diffs do not*. Two gradients from different minibatches can always be summed; two textual edits to the same artifact may be complementary, redundant, or contradictory. Aggregation in AgentDescent is therefore not averaging but a discrete-space optimizer step: staleness filtering, conflict resolution, fusion of complementary edits, a statistical acceptance test under a per-artifact Beta posterior, and a transactional compare-and-swap commit (Section 4). A second disanalogy is equally load-bearing: training code is not modifiable by the gradients it computes, but a self-evolving agent system *can* rewrite its own verifier — so AgentDescent enforces a layered governance model in which safety-critical components are frozen and high-blast-radius artifacts are forced through an oracle (Section 6).

Contributions.

1. **A systems design and a discrete-space aggregator** that map the training stack onto parallel self-evolution — staleness policies over a per-diff lag η , PCGrad-style conflict projection, fusion, Beta-posterior acceptance, dual-branch promotion — with an explicit account of where the analogy must break (Sections 3 and 4).
2. **A three-lever account of speed** (Section 5): overlap the I/O-bound rollouts; remove the round barrier; and — the lever specific to merge-based evolution — *validate less*, because held-out evaluation is the critical path and merging collapses N admission sweeps into one, with *reflective merge* extending the saving to artifacts whose concurrent proposals contradict.
3. **Adaptation as plug-ins, not forks** (Sections 3.4 and 7): every component sits behind a swappable seam, and eighteen published algorithms port onto the engine through exactly two routes — a `strategy` plus optional `aggregator_factory`, or a declarative policy bundle — then run under all three schedulers unchanged.
4. **An honest live-model study** (Section 8): $1.85\times \rightarrow 3.22\times$ effective concurrency from removing the barrier against a true serial control at a pinned budget, -28% model calls from merged validation, an eleven-algorithm scheduler matrix, and an equal-budget merge-vs-fork comparison reported as *not yet separated*, with the diagnosis of why.

2 Related Work

Self-evolving agents span prompt evolution [1, 3], context/playbook evolution [28], skill libraries [20, 30], workflow and harness search [5, 27], self-modifying code [14, 24, 26], evolutionary program search [13, 17], and label-free self-play [6, 22, 29], with reflection [10, 18] as the shared inner proposal mechanism — and almost all publish a *serial* outer loop, which AgentDescent treats as one scheduling choice among three. The systems side borrows its vocabulary from distributed training: the parameter server [9], lock-free and

Table 1: The central analogy. Each row of the training stack has a concrete counterpart in AgentDescent; the final row is a *disanalogy* that the design enforces rather than papers over.

Model training	AgentDescent (parallel self-evolution)
parameter tensor θ	library of Evolvable artifacts
gradient g	Diff + EvidenceCard
parameter server	git-backed, version-vectored Ledger
optimizer step	Aggregator merge decision
per-parameter adaptive LR (Adam)	per-artifact Beta-posterior acceptance test
staleness / decoupled PPO	per-diff lag η + rebase re-verification
partial rollout	straggler detection (ResumeQueue)
EMA / weight averaging	stable/dev dual branch
training code (not self-modifiable)	L0 frozen governance layer

bounded-staleness asynchrony [4, 12, 15], per-parameter adaptivity [8], gradient surgery [25], weight averaging [21], and tensor/pipeline parallelism [7, 19]. Concurrent work on parallel self-evolution (FlashEvolve-style barrier-free loops, CoEvoSkills [2]) shows the throughput premise is a live hypothesis, not consensus; AgentDescent’s contribution is the narrow engineering synthesis — concurrent, staleness-bounded, conflict-resolved diff-level merge over a versioned ledger — plus the harness to measure it.

3 The Training-Stack Analogy and System Design

3.1 Evolvables, diffs, and evidence cards

An `Evolvable` is any versioned artifact the system may modify: a skill document, a prompt, a playbook of bullets, a harness module, a directory of files, or program code. A `Diff` is a keyed set of operations against one or more evolvables — a *strategy* (e.g., `AppendRules`, `KeyedRules`, a per-bullet playbook, or file-path-keyed directory edits) maps a natural-language proposal into diff operations, which is what makes distinct edits fusable and conflicting edits detectable. Every diff carries an `EvidenceCard`: the task, the rollout output, the reward, and the base version the proposing worker observed. The card is the unit of statistical bookkeeping; discarded diffs settle their evidence back into the pool so information is not lost with the proposal.

3.2 The ledger

The `Ledger` is the parameter server: a git-backed store with per-artifact version vectors, compare-and-swap commits, and a dual `dev/stable` branch pair. `dev` receives merges; `stable` is promoted only after K regression-free rounds on `dev` — an EMA-style confirmation in which a new commit *restarts* the clock, so the artifact most likely to be promoted is the one that stopped changing because nothing beat it. Workers read snapshots; the version vector is what makes per-diff staleness η computable at merge time.

3.3 The one required disanalogy

Because diffs do not add, every downstream design choice follows from replacing summation with a decision procedure. Averaging two contradictory edits is not meaningful; ranking and projecting them is. This is why the aggregator (next section) is best understood not as a reducer but as a discrete-space optimizer whose “step” is a merge decision, and why acceptance is statistical — a per-artifact posterior over whether the candidate improves held-out reward — rather than a learning rate.

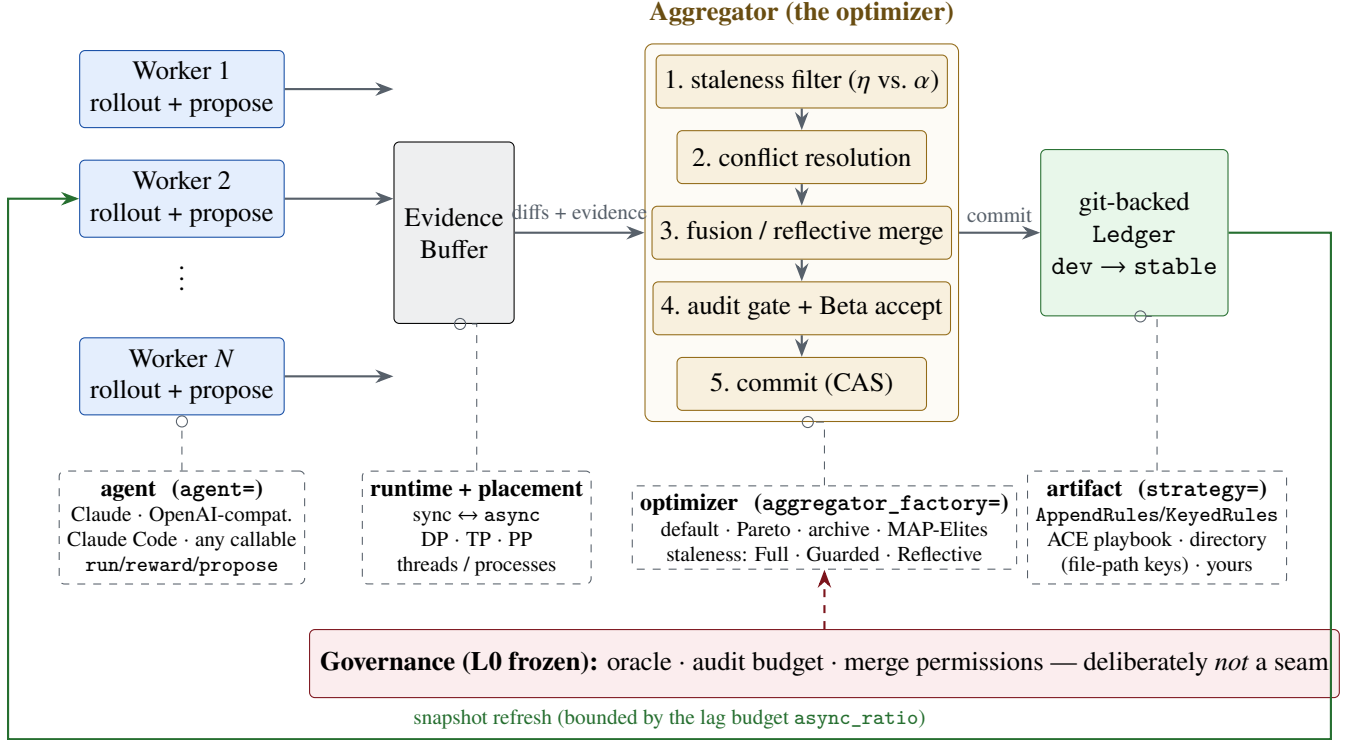


Figure 1: AgentDescent architecture. **Solid, top:** the fixed data path — N workers roll out tasks against ledger snapshots and emit diffs with evidence cards into a thread-safe buffer; a dedicated aggregator thread runs the five-stage merge pipeline and commits winners to the git-backed ledger; the green return edge is the only feedback path, bounded by the lag budget. **Dashed, bottom:** every component on the path is a *pluggable seam*, swapped by one keyword argument of `evolve()` — the model behind a worker, the whole optimizer, the staleness policy, the runtime and placement, and the artifact’s diff encoding. The eighteen ports of Section 7 are exactly such plug-ins. The one deliberate exception is the red governance layer: L0 (oracle, audit budget, merge permissions) is frozen and read-only to the loop, because a self-referential system must not be able to swap out its own judge.

3.4 Everything behind the seam is pluggable

The architecture of Fig. 1 is deliberately a set of *seams*, each swapped by one keyword argument of the single entry point `evolve()`. The **agent layer** is provider-agnostic — any `prompt` $\rightarrow$ `text` callable is a completion, so Claude, OpenAI-compatible endpoints, a tool-using CLI agent, or three plain Python functions (`run/reward/propose`) plug in interchangeably. The **strategy** (`strategy=`) defines what evolves and how proposals become keyed diff operations: append-only rule lists, keyed rule slots, ACE-style playbooks, or whole directories whose state keys are file paths — the optimizer never changes, only the diff encoding does. The **aggregator itself** is replaceable (`aggregator_factory=`): GEPA’s Pareto optimizer, ADAS/DGM-style archives, and OpenEvolve’s MAP-Elites islands all slot in at this seam, with their parent-selection and gate rules extracted as named policy classes at two further seams (selection, acceptance). The **staleness policy**, the **runtime** (`asynchronous=True`), the **parallelism strategy** (DP/TP/PP or a custom one), the **executor** (in-process threads or supervised worker processes), and the **sandbox isolation level** are likewise independent arguments. The eighteen ports of Section 7 are the load test of this claim: none of them forks the engine — each is precisely a bundle of plug-ins at these seams. The single deliberate exception is governance (Section 6): the L0 layer is frozen and exposes no seam, because a self-modifying system must not be able to swap out its own judge.

4 The Aggregator as a Discrete-Space Optimizer

Evidence cards are bucketed by artifact; when a bucket triggers (batch size B or a timeout so cold artifacts do not starve), the aggregator runs one optimizer step (Fig. 1, center): **(1) staleness filter** — per-diff lag $\eta = \max(\text{head} - \text{base})$; $\eta=0$ proceeds, $\eta \leq \alpha$ is rebased and cheaply re-verified, beyond is discarded with its evidence settled back into the pool [4]; **(2) conflict resolution** — contradictions are projected out PCGrad-style [25], keeping the better of the pair on a shared held-out subset; **(3) fusion** — complementary diffs merge into one union candidate [21], with *reflective merge* handling contradictions a keyed union cannot (Section 5.3); **(4) audit gate** — high-blast-radius or low-trust merges are forced through the oracle, which can veto before the acceptance test runs; **(5) statistical acceptance and commit** — the candidate commits (compare-and-swap on dev) only if $P(\Delta > 0) > 1 - \delta$ under a per-artifact Beta posterior over held-out outcomes, the analogue of Adam’s per-parameter adaptivity [8], with δ annealing over versions and a trust region capping diff size; dev promotes to `stable` after K regression-free rounds.

The sharpest objection — two individually good edits, jointly worse — is bounded twice over: the gate scores the *fused* candidate on the full held-out set and refuses a measured regression, and the union is a superset of every single diff, so committing it loses no proposal. An optional *fusion tournament* additionally ranks the union against its singles; it is off by default (one extra held-out sweep per candidate against a recoverable gain) but is the instrument that *measures* the objection, reporting win rate, losing tail, and a contested counter so a run in which fusion never actually fired cannot be misread as a merge win. No headline win rate is published, deliberately: the quantity is a property of the artifact’s key space and proposal overlap, not of the mechanism (Section 8.5 reports it in situ).

5 Why It Is Fast: Three Levers

Wall-clock in a self-evolution loop goes to three places: rollouts, waiting at the barrier, and held-out validation. AgentDescent attacks each with its own lever, and the third — validating less — is the one specific to *merge-based* evolution.

5.1 Lever 1: overlap the rollouts

Agent rollouts are I/O-bound — the worker spends its time waiting on a model — so plain threads already overlap them (measured on real API calls: 5.8× at 8 threads; Section 8.1). The synchronous data-parallel runtime shards tasks across N workers on one snapshot and merges at a round barrier. But the barrier imposes a ceiling no worker count lifts: a round lasts $\mathbb{E}[\max \text{ of } N]$ rather than $\mathbb{E}[\text{latency}]$, and reasoning models have exactly the heavy-tailed latency that maximizes this gap — a short answer and a 2,000-token deliberation are the same call. Between rounds a second serial region appears: the gate evaluates while every worker idles. (For completeness, tensor- and pipeline-parallel decompositions of the artifact library [7, 19] are also available at the same seam.)

5.2 Lever 2: remove the barrier

The barrier-free runtime deletes the round structure: workers continuously roll out and propose, a dedicated aggregator thread continuously merges, and a slow rollout delays only its own diff — the no-barrier discipline of asynchronous RL [4, 15] ported to self-evolution. Its price is **staleness**, made first-class and per-diff (η , computable from the ledger’s version vectors) and bounded twice: the *lag budget* `async_ratio` caps snapshot drift at the source (with backpressure forcing a resync if evidence arrives while nothing commits), and a *staleness policy* disposes of the stale diffs that arrive anyway — **Full** uses them as-is (sound only where diffs compose), **Guarded** rebases within α and discards beyond (safe, but it pays for safety in discarded

rollouts [4]), and **Reflective** always rebases and *re-verifies*, discarding only if the claimed delta no longer holds on the new head — a stale diff is not a wrong diff, only an unproven one, so the policy converts would-be waste back into candidates. One footgun generalizes: the budget is denominated in artifact *versions*, so its right value depends on how much wall-clock a version represents; a run that discards more than half its evidence warns at runtime.

5.3 Lever 3: validate less

The lever the first two cannot reach is the gate itself. Every admission decision costs a sweep over the held-out split, and on a live model this is where the merger’s time goes — profiled on GEPA/HotpotQA, the merge thread’s busy time is *100% gate evaluation* to measurement precision. Serial and fork-based loops pay one sweep per proposal. Merging changes the arithmetic: a round’s N surviving diffs fuse into *one* candidate, so N admission sweeps collapse into one; a stale diff discarded by the staleness filter never reaches the gate and costs *zero* evaluations; and in the barrier-free runtime, cards accumulating during a merge make batches larger rather than merges more numerous. Measured at width 8 on a live model, this cuts model calls 28% against the serial arm (70 vs. 97) while model-seconds per rollout fall $37.9 \rightarrow 23.8$ across the arms (Section 8.2). The remaining gate cost parallelizes independently (`eval_concurrency`), and it saturates at the held-out size — a second knob, orthogonal to worker count.

Fusing requires that proposals *can* fuse, which fails exactly where the artifact’s key space is too coarse. A prompt held in a single instruction slot — GEPA’s artifact shape [1] — makes every pair of concurrent proposals a write–write conflict on one key; a keyed union then degenerates to keeping the better single, i.e. best-of- N selection paying best-of- N ’s validation bill. **Reflective merge** restores the lever: a model writes the *union* of the contradicting proposals — one candidate preserving each proposal’s intent — and that union goes through the same acceptance gate as any candidate, so the model generates and the gate decides. It is the same move as the Reflective staleness policy in the temporal direction (carry intent onto a changed base; let measurement decide), and the contested counter reports whether fusion genuinely fired so the saving is never claimed where only selection happened.

6 Governance by Blast Radius

Artifacts sort into layers by a declared `blast_radius`: **L2** skills and prompts merge under the full asynchronous pipeline; **L1** harnesses and verifiers serialize in-flight changes and force every merge through the oracle; **L0** — the oracle, audit budget, merge permissions, safety constraints — is frozen and read-only to the loop. The frozen layer enforces the second disanalogy of Table 1: without it, a self-referential loop eventually pollutes itself (a verifier that learns to pass itself). The same layering guards executed agent *code*, which runs behind a frozen test suite the candidate cannot rewrite — pristine tests are overlaid after materialization, so weakening them at run time also fails.

7 Adapting Published Algorithms: a Port Is a Plug-In

Because every component sits behind a seam (Section 3.4), porting a published algorithm is not a fork of the engine but a small bundle of plug-ins, and eighteen ports are the demonstration. There are exactly two routes in.

Route 1 — a strategy, and if needed an optimizer. A *benchmark-faithful* port writes a strategy that encodes the paper’s artifact as keyed diffs, and — only when the paper’s search differs from the default pipeline — an `aggregator_factory`. The seven faithful ports are instructive in how little that is: ACE is a playbook strategy whose Curator *is* the default aggregator; GEPA swaps in a per-instance Pareto optimizer;

ADAS and DGM plug in keep-all archives with their own parent selection; EvoSkill a bounded top- K frontier; SkillOpt a strict edit gate; OpenEvolve MAP-Elites islands [11]. Their parent-selection and acceptance rules are extracted as named policy classes at the standard seams, so the next port reuses them.

Route 2 — a declarative policy bundle. The eleven microports and analogues do not even write a class: each is a `MethodPolicy` — selection rule, memory, acceptance shape, genome — over one shared runner, so all eleven inherit the same budget contract, dataset layer, and command line. The shared `--serial` flag runs the *upstream algorithm’s own semantics* (one worker, serial gate) as the control every parallelization claim needs, and a shared rollout budget pins both arms to equal work.

Fidelity is declared, not implied: each port carries a class — *benchmark-faithful*, *mechanism microport*, *self-edit analogue*, *environment analogue*, *inference analogue* — is faithful to the *released code* where it diverges from the paper, and documents any infrastructure boundary rather than hiding it. All eighteen run serial, synchronous, and barrier-free without modification, which is the load test of the seams: the scheduler comparison of Section 8.3 varies *only* the scheduler.

8 Experiments

Every result in this section comes from a *live model on a real dataset* — `deepseek-v4-flash` or `GLM-5.2` (both reasoning models) through an OpenAI-compatible endpoint, on the algorithms’ own benchmarks. We report one typical result per claim; the full measurement tables, together with the exact reproduction command for every number and the stub-latency characterizations that guided the design, live in the repository documentation. Absolute times depend on host and endpoint, so ratios are the reproducible quantity.

8.1 Lever 1 measured: overlap on real API calls

Eight threads over one pool take a real `GLM-5.2` API workload from 48.6 s to 8.3 s (**5.8×**; sub-linear because a heavy-tailed reasoning model finishes when its slowest call does), while the same threads on pure-Python CPU work gain nothing (1.1 $\times$): the speedup tracks how much of a rollout is *waiting*, which for model-backed agents is nearly all of it.

8.2 The flagship: a live model against a true serial control

Levers 2 and 3 are tested together on a faithful port of a published serial algorithm on its own benchmark: GEPA on HotpotQA [1, 23], 16 rollouts pinned on every arm, against a control that is the upstream loop *exactly* — one worker *and* a serial gate, overlapping at precisely 1.00 $\times$. Figure 2: the synchronous arm reaches 1.85 $\times$ effective concurrency, and removing the barrier takes the *same* four workers to 3.22 $\times$ (wall-clock 609 $\rightarrow$ 239 $\rightarrow$ 140 s). Figure 3 repeats this at width 8 and separates the two savings: the synchronous arm converts merging into a *validation* saving — -28% model calls (70 vs. 97), because each round’s surviving diffs fuse into one gate sweep — while the asynchronous arm converts the same width into a *wall-clock* saving (1.92 $\times$). Model-seconds per rollout fall 37.9 $\rightarrow$ 27.7 $\rightarrow$ 23.8 across the arms: the parallel arms are not just faster, they *score less*, which is Lever 3 in the raw data. Endpoint probes sustain 7–10 $\times$ concurrency, so no arm is provider-limited. Two async caveats: with no round boundary to stop at, it overran the pinned budget (19 vs. 16 rollouts), and at the default lag budget it discarded every stale proposal it saw — Lever 3’s zero-cost discards, paid for in proposals.

8.3 Across eleven algorithms: the runtime matrix

The GEPA result could be a property of one algorithm, so the same scheduler comparison — serial, synchronous-parallel, barrier-free — is run over all eleven declarative ports at an equal candidate budget against a live

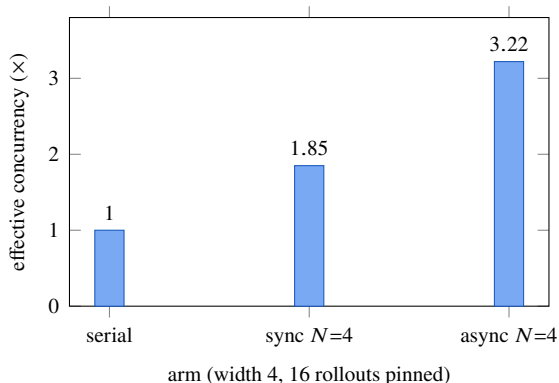


Figure 2: Effective concurrency (model-seconds $\div$ wall-clock) on the faithful GEPA port, HotpotQA, deepseek-v4-flash, 16 rollouts pinned on every arm. The serial control is the published loop exactly (one worker, serial gate: overlap 1.00 $\times$). The round barrier is the whole difference between the parallel arms: the same four workers reach 1.85 $\times$ with it and 3.22 $\times$ without it. Wall-clock: 609 s / 239 s / 140 s.

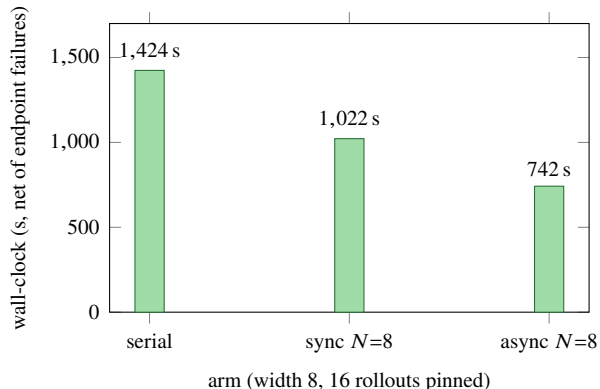


Figure 3: The same comparison at width 8 with GEPA’s own Pareto optimizer, GLM-5.2, wall-clock net of endpoint failures. Model calls: 97 (serial), 70 (sync — reflective merge collapses each round’s admissions into one sweep, -28%), 95 (async). Raw endpoint probes sustain 7–10 $\times$ concurrency, so the measured ladder ($1.31 \times / 1.80 \times / 2.89 \times$) is the engine’s structure, not the provider’s ceiling.

model, with nothing varying but the scheduler. Paired over $n=33$ runs, synchronous-parallel beats serial by a median 1.36 $\times$ end-to-end and 1.89 $\times$ inside the engine window, and *all eleven* methods show a median win. The gap between the two numbers is the fixed cost outside the engine (dataset fetch, final test scoring); the gap between 1.89 $\times$ and the worker count is the barrier plus the serial gate, which is exactly what the barrier-free arm removes in Fig. 2.

8.4 Quality: the ports still learn under the aggressive schedule

Speed is worthless if the ports stop learning under the parallel schedulers, so quality is measured under the most aggressive one. Figure 4 shows the eleven declarative ports, three seeds each against a live model: all improve their held-out test metric, with gains bounded by each benchmark’s headroom exactly as they should be (the three GSM8K ports converge on 0.94–0.97 and separate on *cost*, not quality). The seven benchmark-faithful ports each run end to end on their own benchmark; three typical rows stand for the set: ACE lifts FiNER-139 validation 0.719 $\rightarrow$ 0.766 over 120 rollouts while curating a ten-bullet playbook, DGM’s best archived child fixes held-out vendored bugs at 0.906 against its seed agent’s 0.844 under a frozen real-pytest oracle (its own solver growing from 18 to 79 lines), and SkillOpt lifts a hard SearchQA subset 0.053 $\rightarrow$ 0.211 on validation with 3 of 43 proposed edits accepted. These single-seed runs establish that each port works, not paper-scale comparisons. Finally, the one-call entry point (`evolve_skill`) on 40 real HotpotQA rows lifts held-out exact match from 0.167 to 0.583 across 338 model calls, committing one proposal and rejecting three — the accepted one being the instruction “Respond with only the requested answer, omitting any extra explanation or restatement.”

8.5 Merge-of- N against best-of- N fork: an honest null

Every number above is a throughput number, and throughput cannot distinguish *merging* from *sampling-and-selecting*: N independent forks saturate N workers just as well. The distinguishing quantity is held-out quality at an *equal rollout budget*. The largest such run to date — HotpotQA through GLM-5.2, three arms $\times$ three seeds, budget pinned in rollouts, reflective merge installed so contradicting proposals survive to be fused;

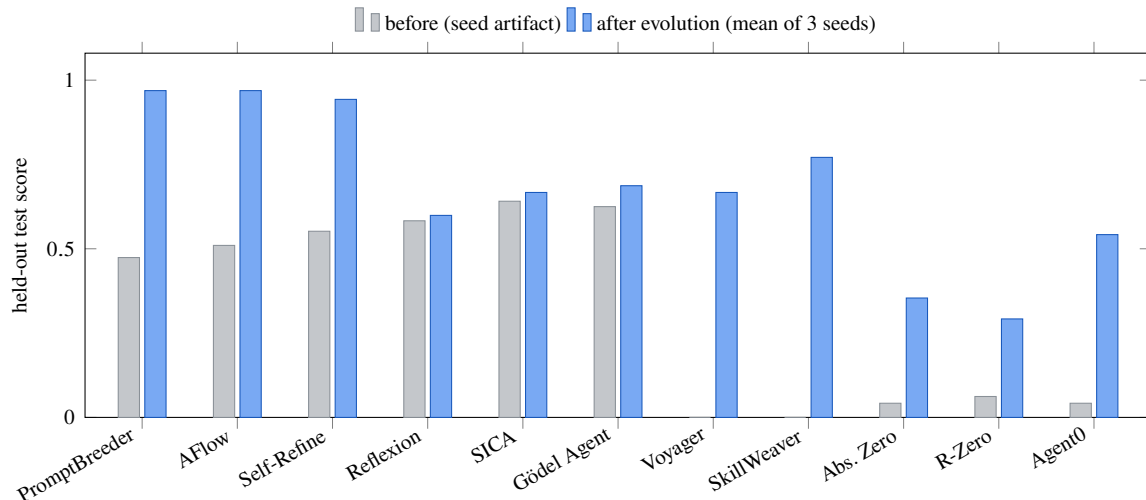


Figure 4: Held-out test score before and after evolution for the eleven microports and analogues, each run against deepseek-v4-flash under the *most aggressive* configuration — barrier-free async pipeline, 8 workers, Full staleness, 80 rollouts, 3 seeds (465–2,069 model calls per seed). All eleven improve. Between 3 and 10 of 240 proposals commit per method — the acceptance gate admits only strict held-out improvements, so sparse acceptance is the shape of the mechanism, not a low yield. Bars are comparable only within a domain: a gain is bounded by the headroom its baseline leaves.

1,298 model calls and 11.3 hours of model time in all — returns a **null**: merge-of-3 posts the highest median test score *and* the widest spread, the intervals overlap, and the framework’s own separation test refuses to call it a win, which is what that test is for. The diagnosis matters more than the verdict: at this budget the fused union was built *twice in the whole experiment*, because most merges saw only a single surviving diff — so this is not evidence that merging does not help; it is an experiment in which merging barely fired. It also confirms the single-key subtlety of Section 5.3 from the other side: without reflective merge, this workload’s merge arm would have been best-of- N selection wearing merging’s name. Larger budgets on multi-key artifacts are the open measurement, and the baseline harness (serial / fork-with-oracle-and-selected / merge, budget-pinned, seeded) ships with the framework.

9 Scope, Limitations, and Honest Reporting

AgentDescent is a research reference implementation. (i) The throughput premise is a testable engineering hypothesis, not consensus, and the equal-budget quality question (Section 8.5) is currently a null at small budgets. (ii) Turn-level checkpoint-and-resume of stragglers and a stratified tail-canary split are not implemented. (iii) Most quality rows carry three seeds and the seven faithful ports one — evidence the mechanisms run, not paper-scale claims; difficulty knobs calibrated on one model did not transfer under a model swap, while benchmark-intrinsic difficulty did. (iv) Several earlier headline numbers in the project’s history proved irreproducible or flatteringly measured, and were re-measured and corrected in place; every number here is generated by a named script. We consider that correction trail part of the contribution: a self-improving system’s evaluation should meet the standard its own acceptance gate imposes on diffs.

10 Conclusion

The deep-learning training stack has a coherent counterpart in agent self-evolution once its one necessary disanalogy — diffs do not add — is taken seriously and merge is built as a discrete-space optimizer. The speed story has three levers, and the distinctive one is validating less: merging turns N admission sweeps into one, reflective merge extends that to artifacts whose proposals contradict, and on a live model the ladder reads $1.85\times$ with the barrier, $3.22\times$ without, and -28% model calls — against a true serial control. Eighteen published algorithms adapt as plug-ins, not forks. Whether merging also buys *quality* over best-of- N at equal budget remains open, and the harness to close that question ships with the framework.

Acknowledgments

The algorithm ports were contributed by chendanyang (the seven benchmark-faithful ports) and cyanneko (OpenEvolve and the eleven microports and analogues).

Reproducibility

All code is at <https://github.com/Birfy/agentdescent> (MIT license; `pip install agentdescent`; the core engine has zero required dependencies). Every measured number in this paper names its generating script: `examples/efficiency.py` (Section 8.1), `bench/matrix_run.py` and `examples/gepa/gepa_prompt_evolution.py` (Section 8.2), `bench/candidate_methods.py` (Sections 8.3 and 8.4), and `bench/baselines_run.py` (Section 8.5). Every port supports `--dry-run` (no API key, no model call) and carries an offline test suite; datasets are fetched from canonical sources by a dependency-free data layer. Absolute timings depend on host and endpoint; the repository documentation states, for each table, whether ratios or absolute values are the reproducible quantity.

References

- [1] Lakshya A Agrawal, Shangyin Tan, Dilara Soylu, Noah Ziem, Rishi Khare, Krista Opsahl-Ong, Arnav Singhvi, Herumb Shandilya, Michael J Ryan, Meng Jiang, et al. GEPA: Reflective prompt evolution can outperform reinforcement learning. *arXiv preprint arXiv:2507.19457*, 2025.
- [2] CoEvoSkills authors. Self-evolving agent skills via co-evolutionary verification. *arXiv preprint arXiv:2604.01687*, 2026.
- [3] Chrisantha Fernando, Dylan Banarse, Henryk Michalewski, Simon Osindero, and Tim Rocktäschel. Promptbreeder: Self-referential self-improvement via prompt evolution. *arXiv preprint arXiv:2309.16797*, 2023.
- [4] Wei Fu, Jiaxuan Gao, Xujie Shen, Chen Zhu, Zhiyu Mei, Chuyi He, Shusheng Xu, Guo Wei, Jun Mei, Jiashu Wang, et al. AReaL: A large-scale asynchronous reinforcement learning system for language reasoning. *arXiv preprint arXiv:2505.24298*, 2025.
- [5] Shengran Hu, Cong Lu, and Jeff Clune. Automated design of agentic systems. *arXiv preprint arXiv:2408.08435*, 2024.
- [6] Chengsong Huang, Wenhao Yu, Xiaoyang Wang, Hongming Zhang, Zongxia Li, Ruosen Fang, Jiabin Huang, Tony Sanders, and Dong Yu. R-Zero: Self-evolving reasoning LLM from zero data. *arXiv preprint arXiv:2508.05004*, 2025.

- [7] Yanping Huang, Youlong Cheng, Ankur Bapna, Orhan Firat, Dehao Chen, Mia Chen, HyukJoong Lee, Jiquan Ngiam, Quoc V Le, Yonghui Wu, et al. GPIPE: Efficient training of giant neural networks using pipeline parallelism. *Advances in Neural Information Processing Systems*, 32, 2019.
- [8] Diederik P Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.
- [9] Mu Li, David G Andersen, Jun Woo Park, Alexander J Smola, Amr Ahmed, Vanja Josifovski, James Long, Eugene J Shekita, and Bor-Yiing Su. Scaling distributed machine learning with the parameter server. In *11th USENIX Symposium on Operating Systems Design and Implementation (OSDI 14)*, pages 583–598, 2014.
- [10] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, et al. Self-refine: Iterative refinement with self-feedback. *Advances in Neural Information Processing Systems*, 36, 2023.
- [11] Jean-Baptiste Mouret and Jeff Clune. Illuminating search spaces by mapping elites. *arXiv preprint arXiv:1504.04909*, 2015.
- [12] Feng Niu, Benjamin Recht, Christopher Ré, and Stephen J Wright. HOGWILD!: A lock-free approach to parallelizing stochastic gradient descent. *Advances in Neural Information Processing Systems*, 24, 2011.
- [13] Alexander Novikov, Ngân Vũ, Marvin Eisenberger, Emilien Dupont, Po-Sen Huang, Adam Zsolt Wagner, Sergey Shirobokov, Borislav Kozlovskii, Francisco J. R. Ruiz, Abbas Mehrabian, et al. AlphaEvolve: A coding agent for scientific and algorithmic discovery. *arXiv preprint arXiv:2506.13131*, 2025.
- [14] Maxime Robeyns, Martin Szummer, and Laurence Aitchison. A self-improving coding agent. *arXiv preprint arXiv:2504.15228*, 2025.
- [15] ROLL Team. ROLL Flash: Accelerating RLVR and agentic training with asynchrony. *arXiv preprint*, 2025.
- [16] Jürgen Schmidhuber. Gödel machines: Self-referential universal problem solvers making provably optimal self-improvements. *arXiv preprint cs/0309048*, 2003.
- [17] Asankhaya Sharma. OpenEvolve: an open-source evolutionary coding agent. <https://github.com/codelion/openevolve>, 2025.
- [18] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. *Advances in Neural Information Processing Systems*, 36, 2023.
- [19] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. Megatron-LM: Training multi-billion parameter language models using model parallelism. *arXiv preprint arXiv:1909.08053*, 2019.
- [20] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. *arXiv preprint arXiv:2305.16291*, 2023.

- [21] Mitchell Wortsman, Gabriel Ilharco, Samir Ya Gadre, Rebecca Roelofs, Raphael Gontijo-Lopes, Ari S Morcos, Hongseok Namkoong, Ali Farhadi, Yair Carmon, Simon Kornblith, et al. Model soups: Averaging weights of multiple fine-tuned models improves accuracy without increasing inference time. In *International Conference on Machine Learning*, pages 23965–23998, 2022.
- [22] Peng Xue, Cheng Wu, Zhen Lin, et al. Agent0: Unleashing self-evolving agents from zero data via tool-integrated reasoning. *arXiv preprint*, 2025.
- [23] Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W Cohen, Ruslan Salakhutdinov, and Christopher D Manning. HotpotQA: A dataset for diverse, explainable multi-hop question answering. In *Proceedings of EMNLP*, 2018.
- [24] Xunjian Yin, Xinyi Wang, Liangming Pan, Xiaojun Wan, and William Yang Wang. Gödel agent: A self-referential agent framework for recursive self-improvement. *arXiv preprint arXiv:2410.04444*, 2024.
- [25] Tianhe Yu, Saurabh Kumar, Abhishek Gupta, Sergey Levine, Karol Hausman, and Chelsea Finn. Gradient surgery for multi-task learning. *Advances in Neural Information Processing Systems*, 33:5824–5836, 2020.
- [26] Jenny Zhang, Shengran Hu, Cong Lu, Robert Lange, and Jeff Clune. Darwin gödel machine: Open-ended evolution of self-improving agents. *arXiv preprint arXiv:2505.22954*, 2025.
- [27] Jiayi Zhang, Jinyu Xiang, Zhaoyang Yu, Fengwei Teng, Xionghui Chen, Jiaqi Chen, Mingchen Zhuge, Xin Cheng, Sirui Hong, Jinlin Wang, et al. AFlow: Automating agentic workflow generation. *arXiv preprint arXiv:2410.10762*, 2024.
- [28] Qizheng Zhang, Changran Hu, Shubhangi Upasani, Boyuan Ma, Fenglu Hong, Vamsidhar Kamanuru, Jay Rainton, Chen Wu, Mengmeng Ji, Hanchen Li, et al. Agentic context engineering: Evolving contexts for self-improving language models. *arXiv preprint arXiv:2510.04618*, 2025.
- [29] Andrew Zhao, Yiran Wu, Yang Yue, Tong Wu, Quentin Xu, Yang Yue, Matthieu Lin, Shenzhi Wang, Qingyun Wu, Zilong Zheng, et al. Absolute zero: Reinforced self-play reasoning with zero data. *arXiv preprint arXiv:2505.03335*, 2025.
- [30] Boyuan Zheng, Michael Y Fatemi, Xiaolong Jin, Zora Zhiruo Wang, Apurva Gandhi, Yueqi Song, Yu Gu, Jayanth Srinivasa, Gaowen Liu, Graham Neubig, et al. SkillWeaver: Web agents can self-improve by discovering and honing skills. *arXiv preprint arXiv:2504.07079*, 2025.